

Python Script Used for Injecting Bit-Flip Faults

```
doc.add_paragraph(f'Registers tested: {"", ".join(registers_to_test)}')
```

```
doc.add_paragraph(f'Bit flipped: {bit_to_flip}')
```

```
doc.add_paragraph(f'Iterations: {iterations}')
```

```
doc.add_paragraph("")
```

```
for i in range(iterations):
```

```
    doc.add_heading(f'Iteration {i+1}', level=1)
```

```
    # Interrupt CPU to safely read/write registers
```

```
    gdbmi.write('interrupt')
```

```
    time.sleep(0.5)
```

```
    for reg in registers_to_test:
```

```
        resp = gdbmi.write(f'-data-evaluate-expression ${reg}')
```

```
        current_val = None
```

```
        for r in resp:
```

```
            if r['type'] == 'result' and 'payload' in r:
```

```
                val_str = r['payload'].get('value')
```

```
                if val_str:
```

```
                    try:
```

```
                        current_val = int(val_str, 16)
```

```
                    except ValueError:
```

```
                        current_val = None
```

```
                    break
```

```
        if current_val is None:
```

```
            doc.add_paragraph(f'[-] Could not read register {reg}')
```

continue

```
flipped_val = current_val ^ (1 << bit_to_flip)
```

```
gdbmi.write(f'set ${reg} = {flipped_val}')
```

```
doc.add_paragraph(
```

```
    f'Register {reg}: initial=0x{current_val:x}, flipped bit {bit_to_flip} → 0x{flipped_val:x}>    )
```

```
doc.add_paragraph("")
```

```
time.sleep(delay_seconds)
```

```
report_path = '/mnt/f/loggins/bit_flip_fault_report.docx'
```

```
doc.save(report_path)
```

```
print(f'[+] Completed {iterations} iterations. Report saved at {report_path}')
```

```
gdbmi.exit()
```

Bit Flip Fault Injection Report

Test started: 2025-05-16 14:10:59.219081

Registers tested: x10, x11, x12

Bit flipped: 3

Iterations: 50

Iteration 1

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x2279604224, flipped bit 3 → 0x227960422c

Register x12: initial=0x7, flipped bit 3 → 0xf

Iteration 2

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x148065239596, flipped bit 3 → 0x14806523959e

Register x12: initial=0x15, flipped bit 3 → 0x1d

Iteration 3

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x22541685200286, flipped bit 3 → 0x2254168520028e

Register x12: initial=0x29, flipped bit 3 → 0x21

Iteration 4

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x9662604907643534, flipped bit 3 → 0x966260490764353c

Register x12: initial=0x33, flipped bit 3 → 0x3b

Iteration 5

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x59, flipped bit 3 → 0x51

Iteration 6

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x81, flipped bit 3 → 0x89

Iteration 7

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x137, flipped bit 3 → 0x13f

Iteration 8

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x319, flipped bit 3 → 0x311

Iteration 9

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x785, flipped bit 3 → 0x78d

Iteration 10

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x1933, flipped bit 3 → 0x193b

Iteration 11

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x6459, flipped bit 3 → 0x6451

Iteration 12

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x25681, flipped bit 3 → 0x25689

Iteration 13

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x153225, flipped bit 3 → 0x15322d

Iteration 14

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x1389101, flipped bit 3 → 0x1389109

Iteration 15

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x20484361, flipped bit 3 → 0x20484369

Iteration 16

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x541606761, flipped bit 3 → 0x541606769

Iteration 17

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x22571673449, flipped bit 3 → 0x22571673441

Iteration 18

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x2359839634497, flipped bit 3 → 0x235983963449f

Iteration 19

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x621877867529375, flipped bit 3 → 0x62187786752937d

Iteration 40

Register x10: initial=0x8, flipped bit 3 $\rightarrow$ 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 41

Register x10: initial=0x0, flipped bit 3 $\rightarrow$ 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 42

Register x10: initial=0x8, flipped bit 3 $\rightarrow$ 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 43

Register x10: initial=0x0, flipped bit 3 $\rightarrow$ 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 44

Register x10: initial=0x8, flipped bit 3 $\rightarrow$ 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 45

Register x10: initial=0x0, flipped bit 3 $\rightarrow$ 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 $\rightarrow$ 0x441783189658309509

Iteration 46

Register x10: initial=0x8, flipped bit 3 $\rightarrow$ 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 47

Register x10: initial=0x0, flipped bit 3 $\rightarrow$ 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 48

Register x10: initial=0x8, flipped bit 3 $\rightarrow$ 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 49

Register x10: initial=0x0, flipped bit 3 → 0x8

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Iteration 50

Register x10: initial=0x8, flipped bit 3 → 0x0

Register x11: initial=0x-7610414553529830084, flipped bit 3 → 0x-761041455352983008c

Register x12: initial=0x441783189658309501, flipped bit 3 → 0x441783189658309509

Python Script Used for Injecting Stuck-at Faults

```
from pygdbmi.gdbcontroller import GdbController

from docx import Document

from datetime import datetime

import time


registers_to_test = ['x10', 'x11', 'x12'] # Registers to inject faults into
iterations_per_mode = 25 # 25 stuck-at-1 + 25 stuck-at-0
delay_seconds = 2


# Setup GDB MI controller without timeout_sec (fixes your error)
gdbmi = GdbController(command=["gdb-multiarch", "--nx", "--interpreter=mi3"])
gdbmi.write('-target-select remote localhost:1234')
time.sleep(1)


doc = Document()
doc.add_heading('Stuck-At Fault Injection Report', 0)
doc.add_paragraph(f'Test started: {datetime.now()}')
doc.add_paragraph(f'Registers tested: {"", ".join(registers_to_test)}')
doc.add_paragraph(f'Iterations per mode: {iterations_per_mode}')
doc.add_paragraph('Fault modes: stuck-at-1 then stuck-at-0')
doc.add_paragraph("")


def safe_interrupt():
    for attempt in range(3):
        try:
            resp = gdbmi.write('interrupt')
            return resp
        except Exception:
            time.sleep(1)
    raise Exception("GDB did not respond to interrupt after retries.")


# Run stuck-at-1 faults
for i in range(iterations_per_mode):
    doc.add_heading(f'Iteration stuck-at-1: {i+1}', level=1)
```

```

safe_interrupt()

time.sleep(0.5)

for reg in registers_to_test:

    resp = gdbmi.write(f'-data-evaluate-expression ${reg}')

    current_val = None

    for r in resp:

        if r['type'] == 'result' and 'payload' in r:

            val_str = r['payload'].get('value')

            if val_str:

                try:

                    current_val = int(val_str, 16)

                except ValueError:

                    current_val = None

            break

    if current_val is None:

        doc.add_paragraph(f'[-] Could not read register {reg}')

        continue

    stuck_at_val = current_val | (1 << 3) # Stuck bit 3 at 1

    gdbmi.write(f'set ${reg} = {stuck_at_val}')

    doc.add_paragraph(f'Register {reg}: initial=0x{current_val:x}, stuck-at-1 bit 3 → 0x{stuck_at_val:x}')
doc.add_paragraph("")

time.sleep(delay_seconds)

```

Run stuck-at-0 faults

```

for i in range(iterations_per_mode):

    doc.add_heading(f'Iteration stuck-at-0: {i+1}', level=1)

    safe_interrupt()

    time.sleep(0.5)

    for reg in registers_to_test:

        resp = gdbmi.write(f'-data-evaluate-expression ${reg}')

        current_val = None

        for r in resp:

            if r['type'] == 'result' and 'payload' in r:

                val_str = r['payload'].get('value')

                if val_str:

```

```

try:
    current_val = int(val_str, 16)

except ValueError:
    current_val = None

break

if current_val is None:
    doc.add_paragraph(f'[-] Could not read register {reg}')
    continue

stuck_at_val = current_val & ~(1 << 3) # Stuck bit 3 at 0

gdbmi.write(f'set ${reg} = {stuck_at_val}')

doc.add_paragraph(f'Register {reg}: initial=0x{current_val:x}, stuck-at-0 bit 3 → 0x{stuck_at_val:x}')
doc.add_paragraph("")

time.sleep(delay_seconds)


report_path = '/mnt/f/loggins_stuck_at/stuck_at_fault_report.docx'

doc.save(report_path)

print(f'[+] Completed stuck-at faults. Report saved at {report_path}')

gdbmi.exit()

```

Stuck-At Fault Injection Report

Test started: 2025-05-16 18:28:34.766529

Registers tested: x10, x11, x12

Iterations per mode: 25

Fault modes: stuck-at-1 then stuck-at-0

Iteration stuck-at-1: 1

Register x10: initial=0x0, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x0, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x0, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 2

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 3

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 4

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 5

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 6

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 7

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 8

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 9

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 20

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 21

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 22

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 23

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 24

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-1: 25

Register x10: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x11: initial=0x8, stuck-at-1 bit 3 → 0x8

Register x12: initial=0x8, stuck-at-1 bit 3 → 0x8

Iteration stuck-at-0: 1

Register x10: initial=0x8, stuck-at-0 bit 3 → 0x0

Register x11: initial=0x8, stuck-at-0 bit 3 → 0x0

Register x12: initial=0x8, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 2

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 3

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 4

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 5

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 6

Register x10: initial=0x0, stuck-at-0 bit 3 $\rightarrow$ 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 $\rightarrow$ 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 $\rightarrow$ 0x0

Iteration stuck-at-0: 7

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 8

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 9

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 10

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 11

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 12

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 13

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 14

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0
 Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Iteration stuck-at-0: 25

Register x10: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x11: initial=0x0, stuck-at-0 bit 3 → 0x0

Register x12: initial=0x0, stuck-at-0 bit 3 → 0x0

Python Script Used for Injecting Instruction Corruption Faults

```
from pygdbmi.gdbcontroller import GdbController

from docx import Document

from datetime import datetime

import time


instruction_address = 0x80000000

iterations = 50

bit_to_flip = 3

delay_seconds = 2


# Start GDB

gdbmi = GdbController(command=["gdb-multiarch", "--nx", "--interpreter=mi3"])

gdbmi.write("-target-select remote localhost:1234")

time.sleep(1)


# Prepare report

doc = Document()

doc.add_heading("Instruction Corruption Fault Injection Report", 0)

doc.add_paragraph(f"Test started: {datetime.now()}")

doc.add_paragraph(f"Instruction address: 0x{instruction_address:x}")

doc.add_paragraph(f"Bit flipped: {bit_to_flip}")

doc.add_paragraph(f"Iterations: {iterations}")

doc.add_paragraph("")


# Function to safely interrupt target

def safe_interrupt():

    for _ in range(3):

        try:

            gdbmi.write("interrupt")
```

```

        return

    except Exception:

        time.sleep(1)

# Run fault injection

for i in range(iterations):

    doc.add_heading(f"Iteration {i+1}", level=1)

    safe_interrupt()

    time.sleep(0.5)


# Read 4 bytes from the instruction address (big endian word)

read_resp = gdbmi.write(

    f'-data-read-memory 0x{instruction_address:x} x 1 4 4 1'

)


original = None

for r in read_resp:

    if r["type"] == "result" and "payload" in r:

        mem = r["payload"].get("memory")

        if mem and isinstance(mem, list):

            contents = mem[0].get("contents")

            if contents:

                try:

                    original = int(contents, 16)

                except:

                    original = None

            break

if original is None:

    doc.add_paragraph(f"[-] Could not read instruction at 0x{instruction_address:x}")

    doc.add_paragraph("")

```

```
time.sleep(delay_seconds)
```

```
continue
```

```
# Flip the bit
```

```
corrupted = original ^ (1 << bit_to_flip)
```

```
# Write the corrupted instruction back
```

```
gdbmi.write(f"set {{int}}0x{instruction_address:x} = {corrupted}")
```

```
doc.add_paragraph(
```

```
    f"Instruction at 0x{instruction_address:x}: original=0x{original:08x}, bit {bit_to_flip} flip>  )
```

```
doc.add_paragraph("")
```

```
time.sleep(delay_seconds)
```

```
# Save the report
```

```
report_path = "/mnt/f/loggins_instruction_corruption/instruction_corruption_fault_report.docx"
```

```
doc.save(report_path)
```

```
gdbmi.exit()
```